

Estruturas de Dados

Árvores AVL

- Características
- Rotações
- Algoritmos

Histórico

- Inventada em por **Georgy Maximovich Adelson-Velskii** (*Геóргий Макси́мович Адельсо́н-Вéльский*) e **Yevgeniy Mikhailovich Landis** (*Евгений Миха́йлович Ландис*) e publicada em: "An algorithm for the organization of information. Trans. of Akademiya Nauk SSSR. Doklady, 1962, v. 146, no. 2, pp. 263-266 "



Georgy Maximovich
Adelson-Velskii
(Samara, Rússia, 1922)



Yevgeniy Mikhailovich
Landis (Kharkov,
Ucrânia, 1921)

Características

- Manter uma árvore binária de busca balanceada sob a presença de constantes inserções e deleções é **ineficiente**.
- Para contornar esse problema foi criada a árvore **AVL** (*Adelson-Velskii e Landis*).
- A árvore AVL é uma árvore binária com uma **condição de balanço**, porém não completamente balanceada.
- Árvores AVL permitem inserção / deleção e **rebalanceamento** aceitavelmente rápidos.

Características

Critério de balanceamento:

- dado um nodo qualquer, uma árvore está **AVL-balanceada** se as alturas das duas subárvores deste nodo diferem de, no máximo, 1.

Para rebalancear uma árvore após uma inserção, são utilizadas **rotações** de subárvores:

- rotações simples em muitos casos;
- rotações duplas em alguns.

Características

Estrutura de um nodo na árvore AVL

```
Estrutura nodoAVL {  
    info      : tipo_info;  
    esq       : *nodoAVL;  
    dir       : *nodoAVL;  
    alt       : inteiro;  
};
```

- O campo **alt** deve ser atualizado recursivamente quando um nodo for inserido ou deletado.

Características

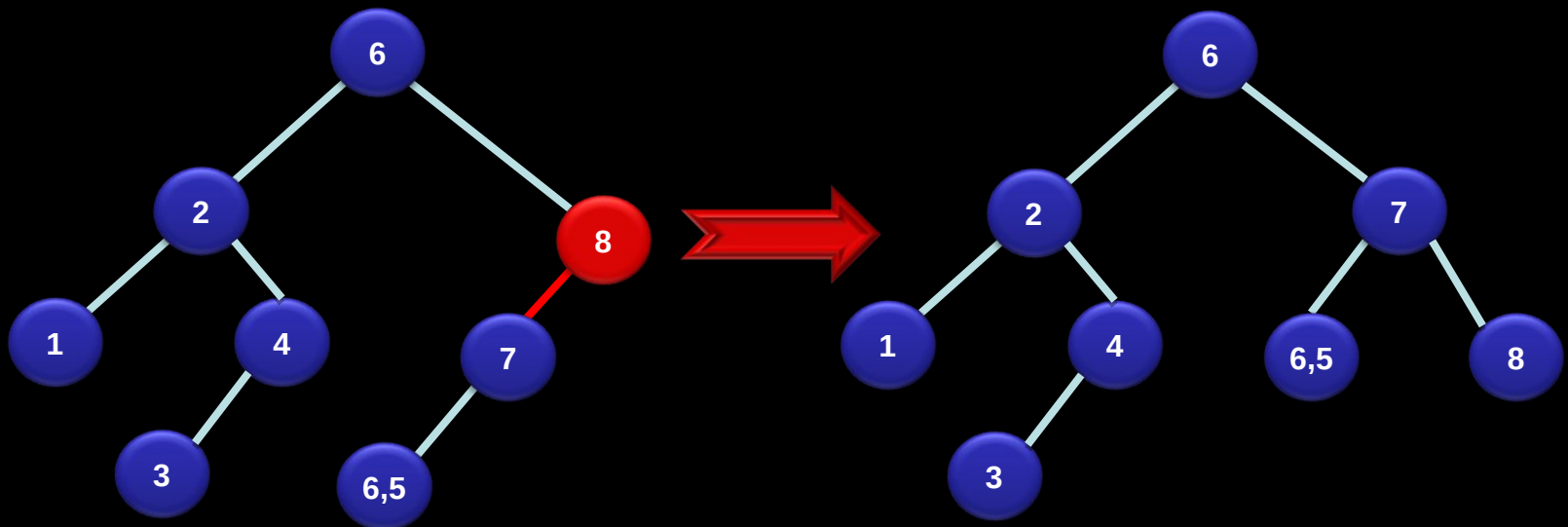
- Para o retorno da altura de uma subárvore necessitamos de uma função especial
 - um nodo pode não ter um ou ambos os filhos
 - embre-se que a pesquisa para garantir a condição-AVL é realizada perguntando-se a altura das duas subárvores

```
inteiro altura( subárvore *nodoAVL)
início
    se (subárvore for NULO)
        retorne -1; /* A altura de uma subárvore
            inexistente é definida como -1 */
    senão
        retorne subárvore->alt;
    fim se
fim
```

Inclusão com rotação

Exemplo de rebalanceamento:

- o nodo com chave **6.5** desequilibrou a árvore. Com a rotação da subárvore em torno do nodo **7**, rebalanceamos.



Inclusão com rotação

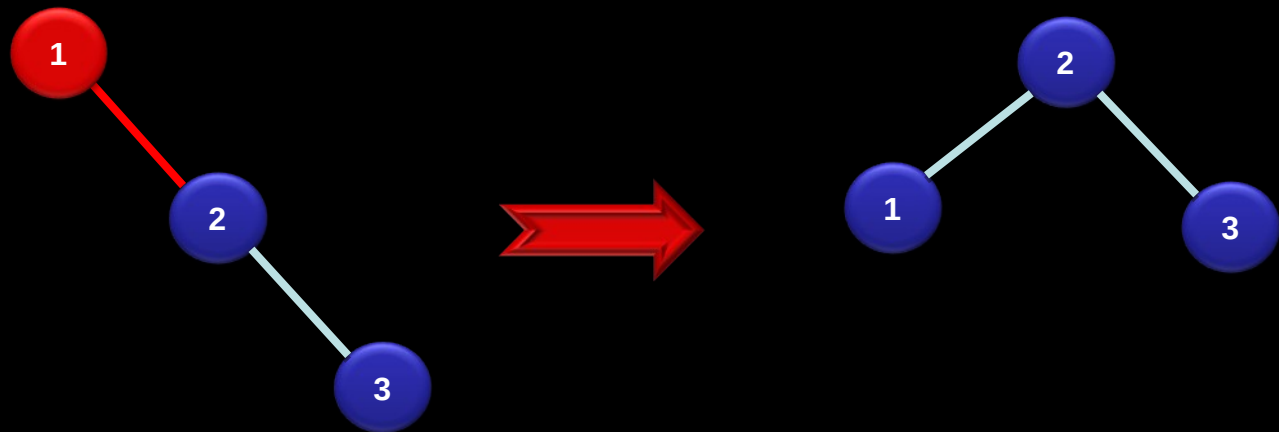
Algoritmo básico:

- partimos do nodo inserido e subimos a árvore;
- atualizamos a informação do balanceamento em cada nodo (na árvore AVL, cada nodo conhece a sua altura);
- se chegamos à raiz sem encontrar nada errado, terminamos;
- caso encontremos um nodo desbalanceado ($|alt_{esq} - alt_{dir}| < 2$ ferida), realizamos a rotação no primeiro nodo desbalanceado encontrado.
 - No exemplo anterior, isto significa que, depois da inserção de 6.5, o nodo 8 ficou desbalanceado.

Exemplo: criação de uma árvore AVL com os números de 1 a 7

O primeiro problema ocorre quando inserimos o **3**. A **condição-AVL** foi violada.

- executamos uma rotação simples entre a raiz (cuja condição-AVL está violada) e seu filho da direita.

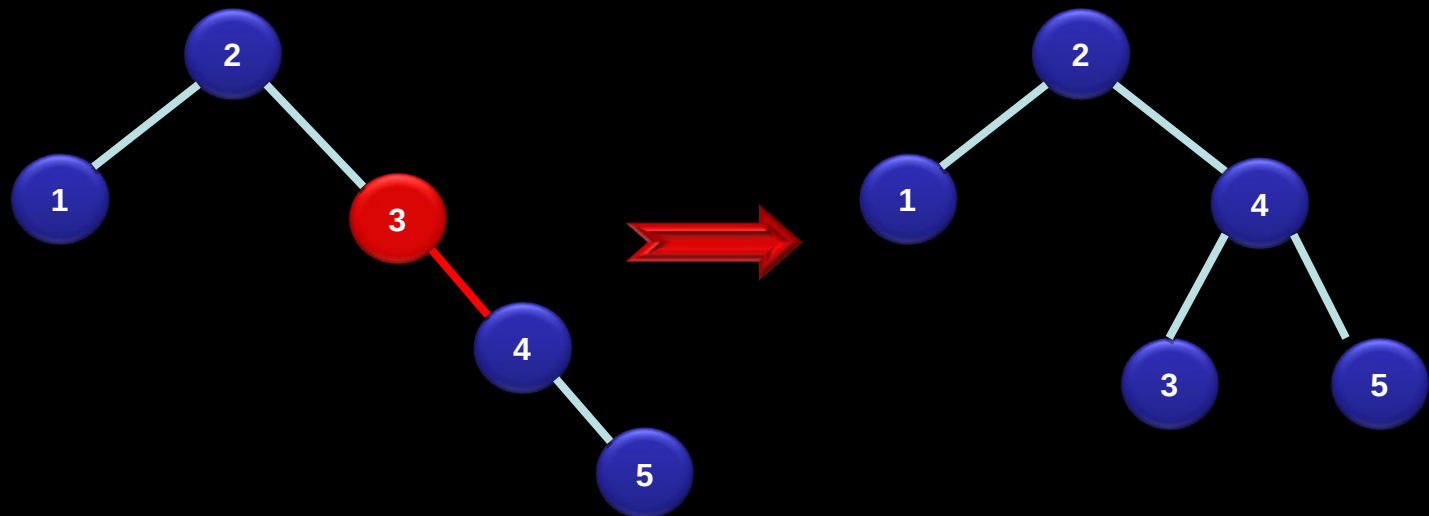


Exemplo: criação de uma árvore AVL com os números de 1 a 7

Inserimos o elemento 4 sem problemas.

Inserimos o elemento 5: violação em 3.

- Mesmo processo da rotação anterior será seguido.
- **Importantíssimo**: observe-se que 2 precisa ser notificado que seu filho agora é o nodo com chave 4.



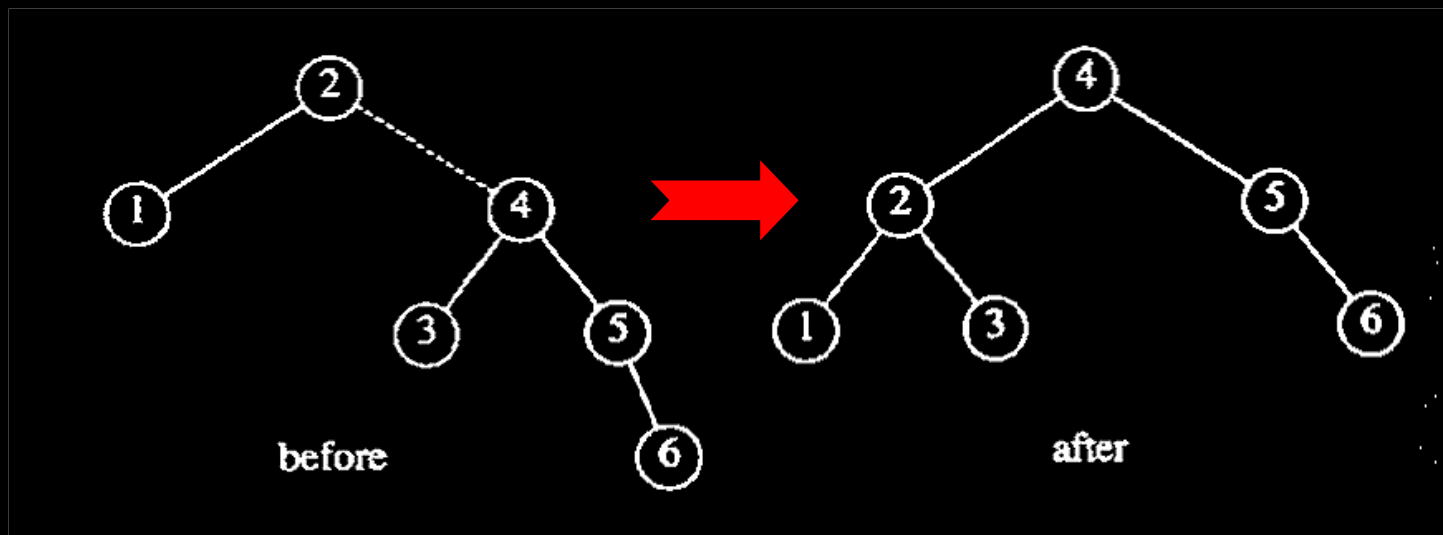
Exemplo: criação de uma árvore AVL com os números de 1 a 7

A inclusão do elemento **6** desequilibra a raiz:

- subárvore direita tem altura 0;
- subárvore esquerda tem altura 2.

Rotacionamos na raiz entre **2** e **4**:

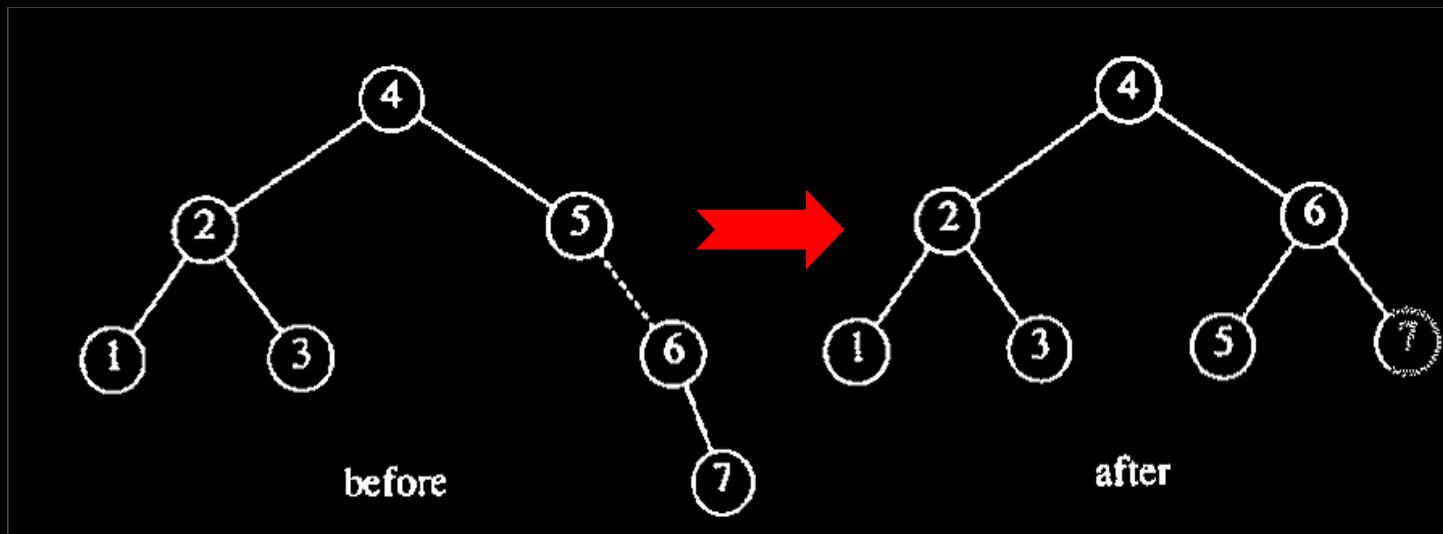
- transformamos **2** em filho de **4**;
- subárvore esquerda original de **4** é agora subárvore direita de **2**. Condição de ordem da árvore de busca é mantida.



Exemplo: criação de uma árvore AVL com os números de 1 a 7

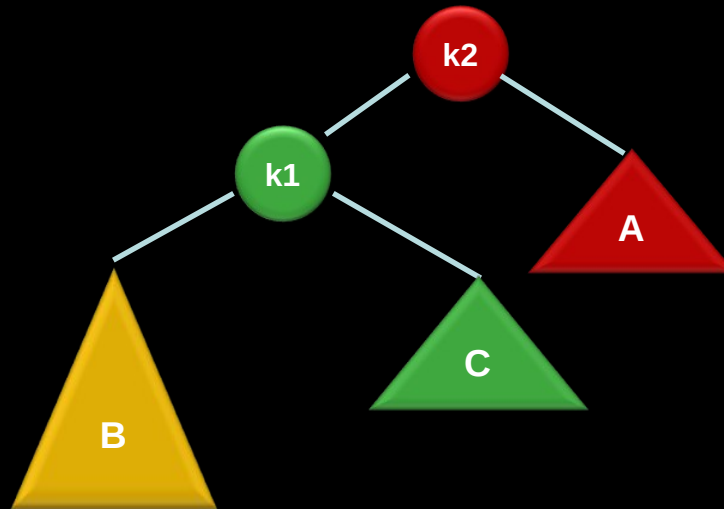
A inclusão de um nodo com chave **7** causa uma rotação como já havíamos visto antes:

- **5** fica desequilibrado;
- rotacionamos em **6**.



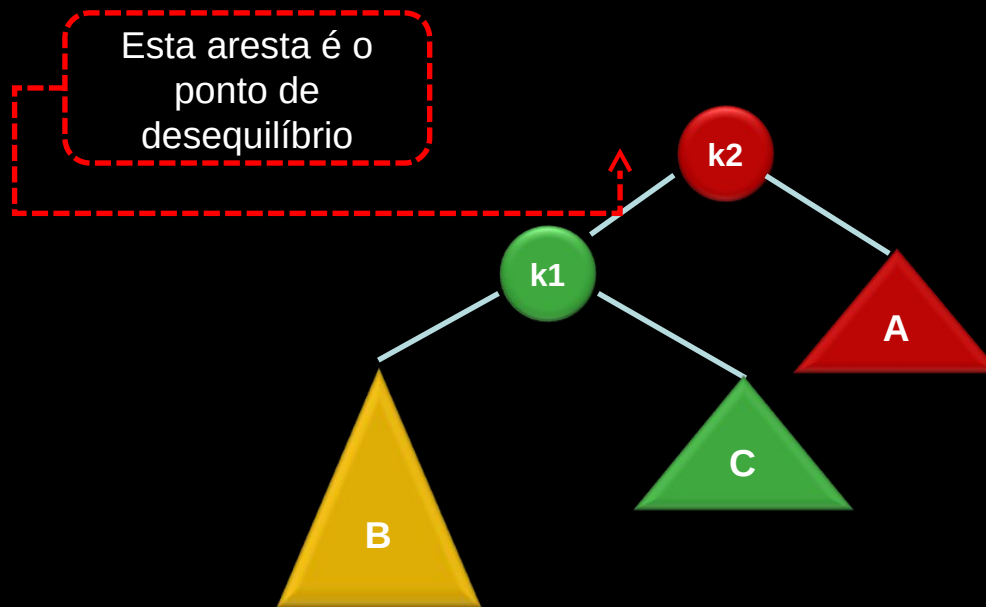
Inclusão com rotação simples à direita

- Corresponde a uma rotação entre os nodos **k2** e seu filho da esquerda, **k1**, envolvendo as subárvores **A**, **B** e **C**



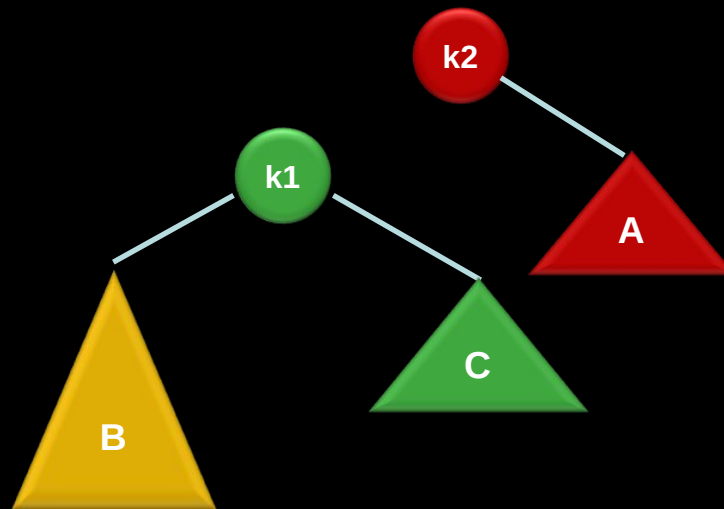
Inclusão com rotação simples à direita

- Corresponde a uma rotação entre os nodos **k2** e seu filho da esquerda, **k1**, envolvendo as subárvores **A**, **B** e **C**



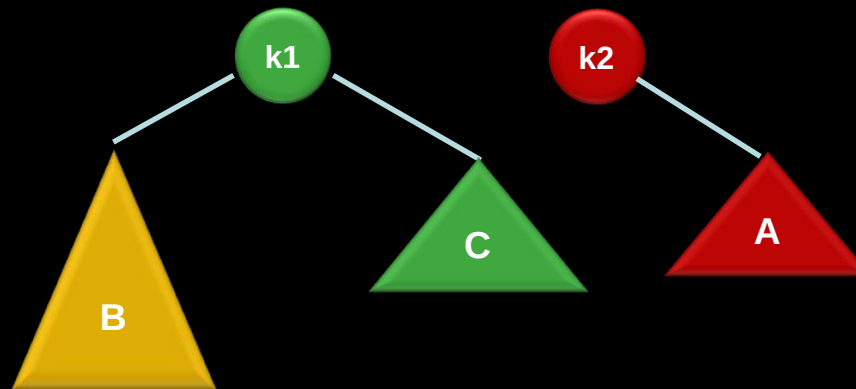
Inclusão com rotação simples à direita

- Corresponde a uma rotação entre os nodos **k2** e seu filho da esquerda, **k1**, envolvendo as subárvores **A**, **B** e **C**



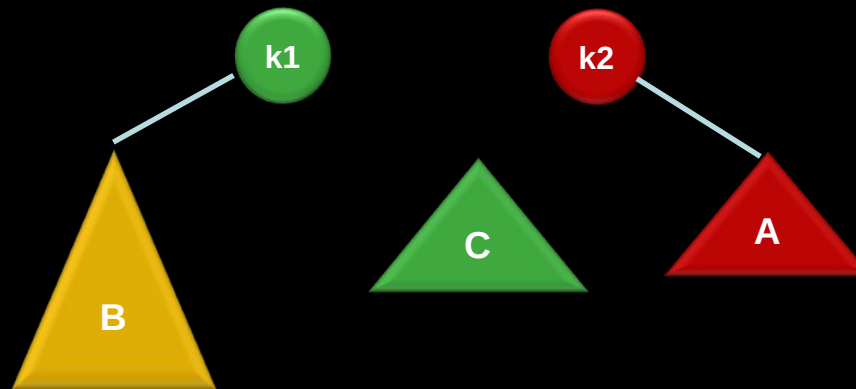
Inclusão com rotação simples à direita

- Corresponde a uma rotação entre os nodos **k2** e seu filho da esquerda, **k1**, envolvendo as subárvores **A**, **B** e **C**



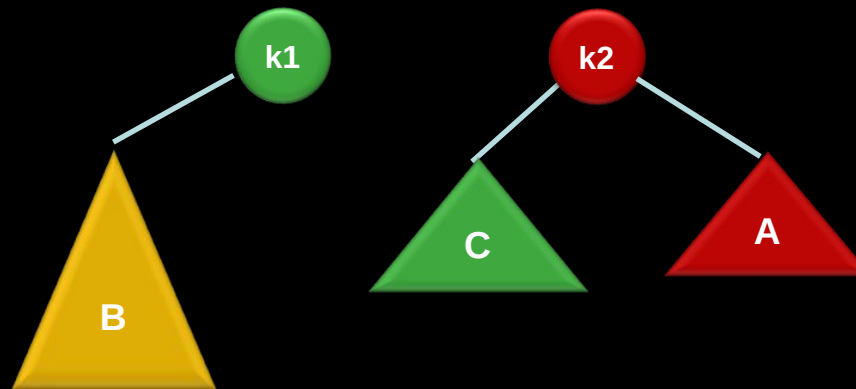
Inclusão com rotação simples à direita

- Corresponde a uma rotação entre os nodos **k2** e seu filho da esquerda, **k1**, envolvendo as subárvores **A**, **B** e **C**



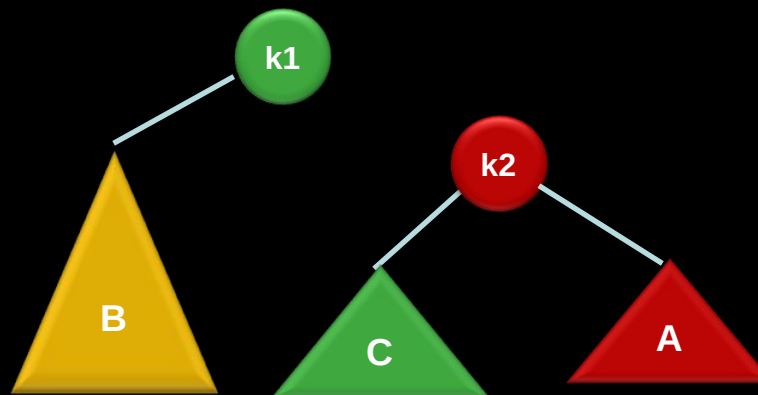
Inclusão com rotação simples à direita

- Corresponde a uma rotação entre os nodos **k2** e seu filho da esquerda, **k1**, envolvendo as subárvores **A**, **B** e **C**



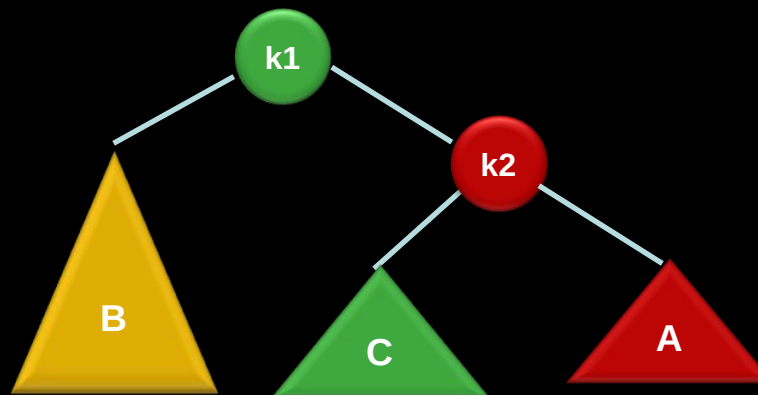
Inclusão com rotação simples à direita

- Corresponde a uma rotação entre os nodos **k2** e seu filho da esquerda, **k1**, envolvendo as subárvores **A**, **B** e **C**



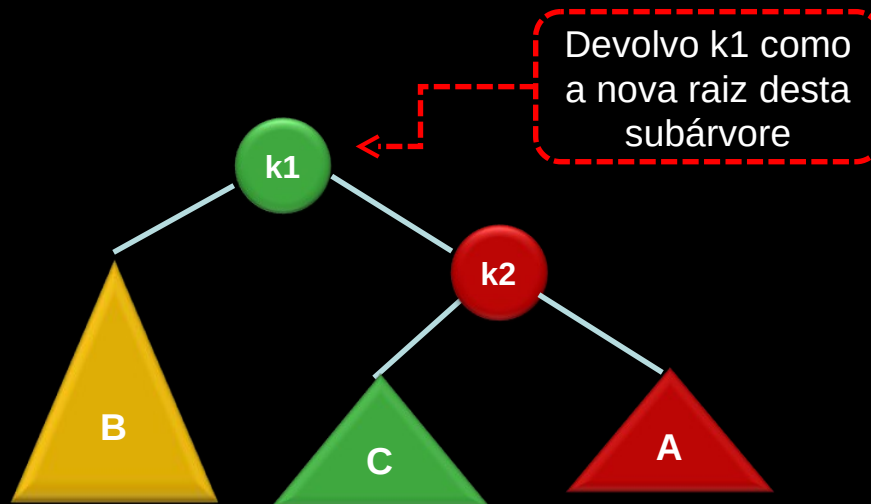
Inclusão com rotação simples à direita

- Corresponde a uma rotação entre os nodos **k2** e seu filho da esquerda, **k1**, envolvendo as subárvores **A**, **B** e **C**



Inclusão com rotação simples à direita

- Corresponde a uma rotação entre os nodos **k2** e seu filho da esquerda, **k1**, envolvendo as subárvores **A**, **B** e **C**



Inclusão com rotação simples à direita

```
nodoAVL *simp_roda_dir(k2 *nodoAVL)
```

```
    variáveis locais
```

```
    k1 : *nodoAVL;
```

```
início
```

```
    k1 <- k2->esq;
```

```
    k2->esq <- k1->dir;
```

```
    k1->dir <- k2;
```

```
    /* atualize alturas */
```

```
    k2->alt <- max(altura(k2->esq), altura(k2->dir)) + 1;
```

```
    k1->alt <- max(altura(k1->esq), k2->alt) + 1;
```

```
    retorne k1; /* nova raiz da subárvore */
```

```
fim
```

Inclusão com rotação simples à esquerda

```
nodoAVL *simp_roda_esq(k2 *nodoAVL)
```

```
    variáveis locais
```

```
        k1 : *nodoAVL;
```

```
início
```

```
    k1 <- k2->dir;
```

```
    k2->dir <- k1->esq;
```

```
    k1->esq <- k2;
```

```
    /* atualize alturas */
```

```
    k2->alt <- max(altura(k2->dir), altura(k2->esq)) + 1;
```

```
    k1->alt <- max(altura(k1->dir), k2->alt) + 1;
```

```
    retorne k1; /* nova raiz da subárvore */
```

```
fim
```

Inclusão com rotação dupla

O algoritmo descrito até agora tem um problema:

- existem casos onde ele não basta para consertar a árvore após uma inclusão ou exclusão.

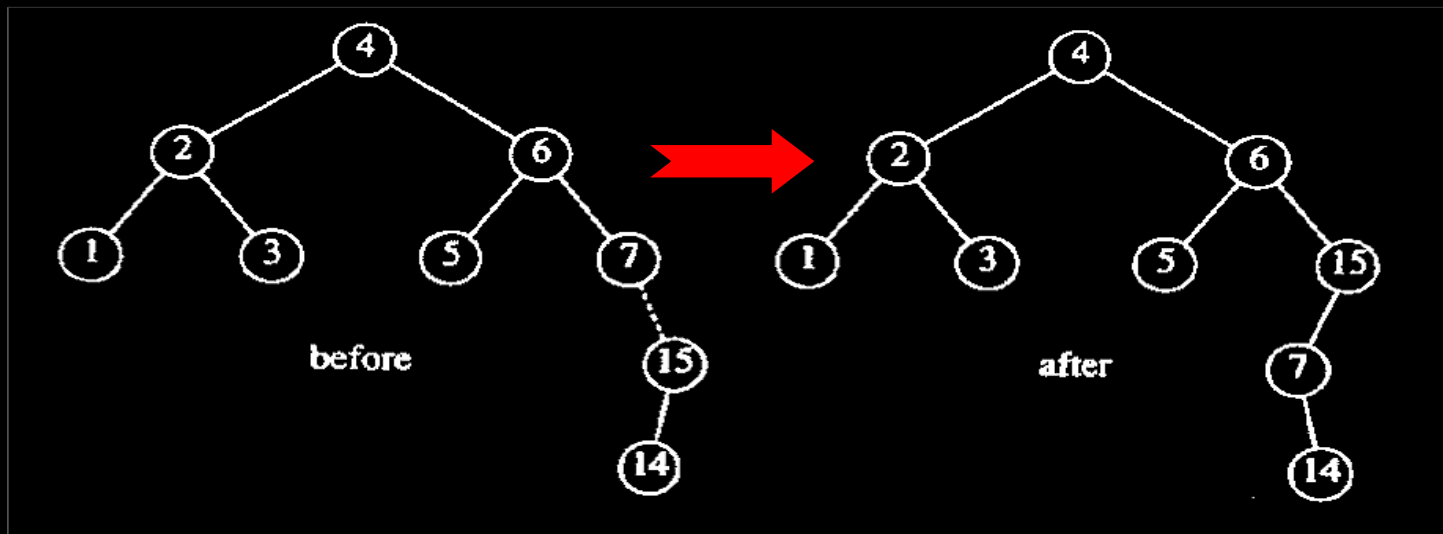
Exemplo: inserimos as chaves 8 a 15, em ordem inversa, na árvore anterior.

- a inserção de 15 não provoca problemas, nem desbalanceia a árvore;
- a inserção de 14, por sua vez, faz o rebalanceamento falhar.

Inclusão com rotação dupla

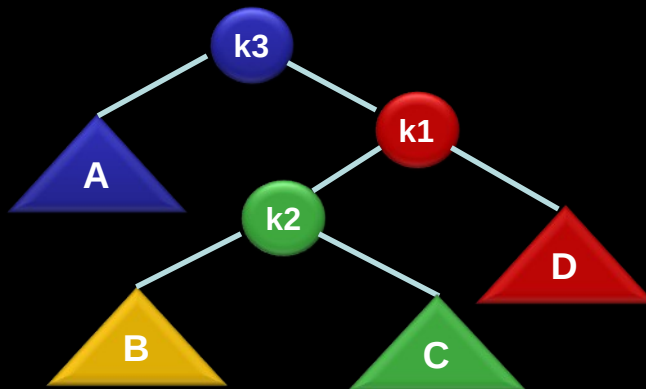
O problema que surge aqui é que tanto **7** como **14** são candidatos à subárvore esquerda de **15**.

Neste caso, necessitamos de uma dupla rotação.



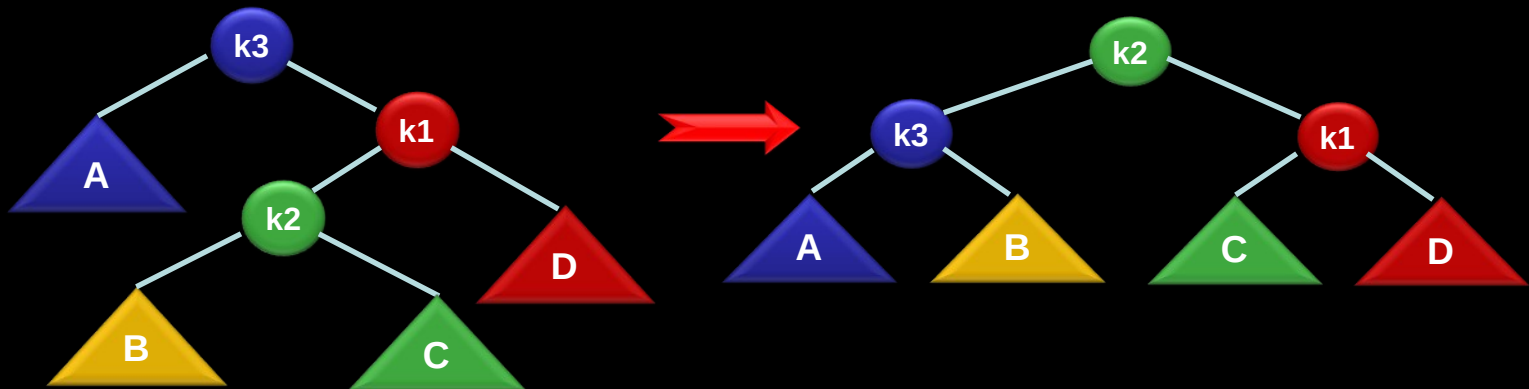
Rotação dupla à esquerda

- O processo é semelhante à rotação simples, só que envolve 4 subárvores: A, B, C e D
 - corresponderia a uma rotação efetuada primeiro entre k1 e k2 e depois entre k2 e k3.



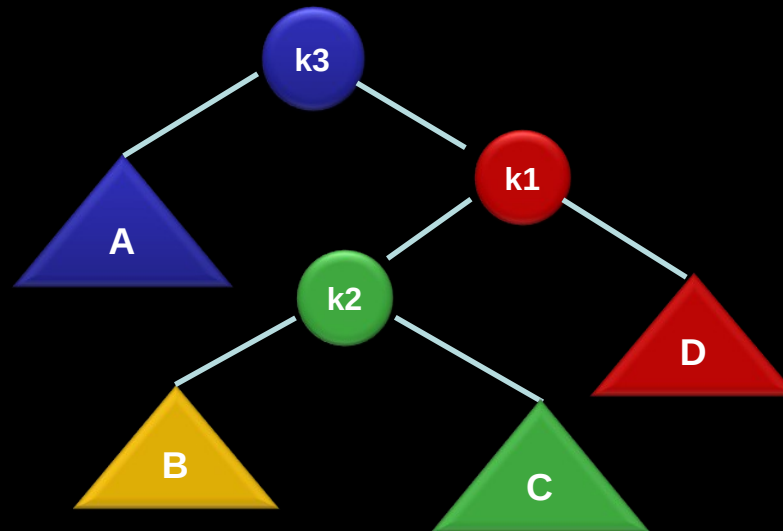
Rotação dupla à esquerda

- O processo é semelhante à rotação simples, só que envolve 4 subárvores: **A**, **B**, **C** e **D**
 - corresponderia a uma rotação efetuada primeiro entre **k1** e **k2** e depois entre **k2** e **k3**.



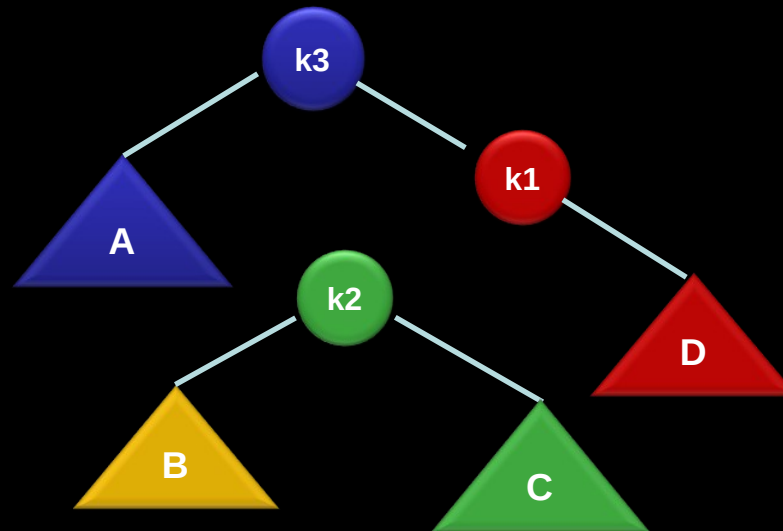
Rotação dupla à esquerda

- P1: Rotação entre k1 e k2



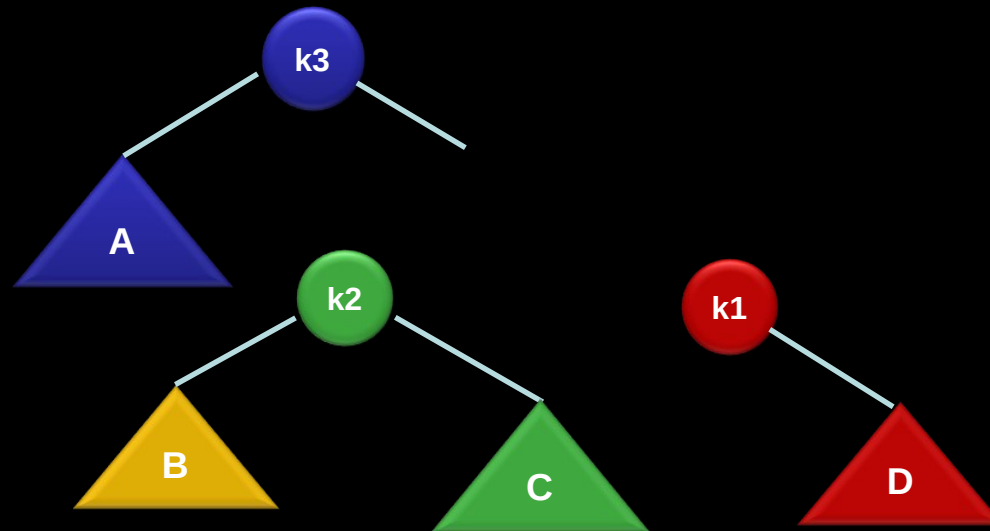
Rotação dupla à esquerda

- P1: Rotação entre k1 e k2



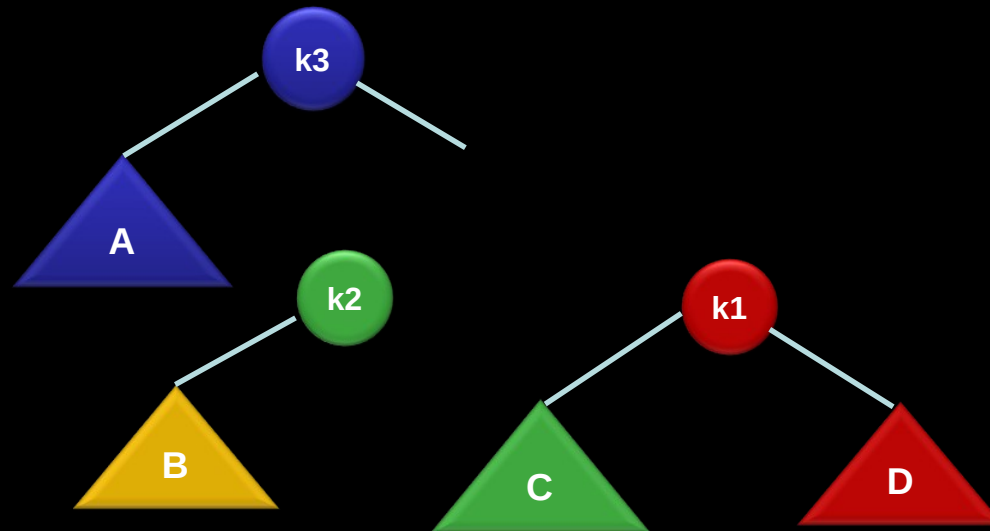
Rotação dupla à esquerda

- P1: Rotação entre k1 e k2



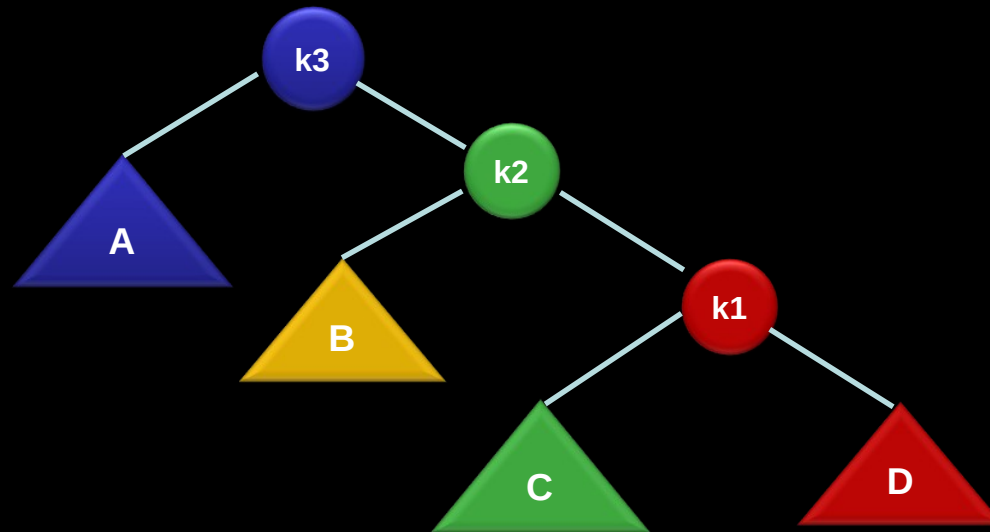
Rotação dupla à esquerda

- P1: Rotação entre k1 e k2



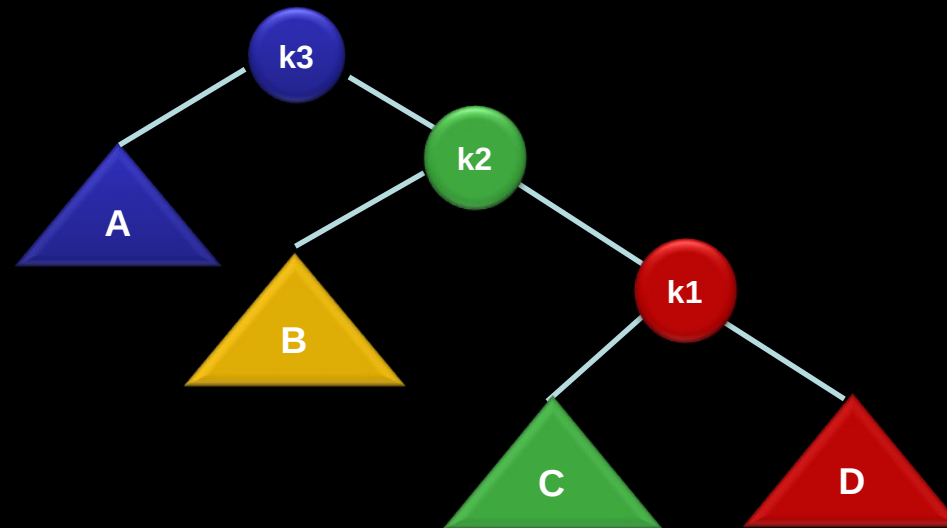
Rotação dupla à esquerda

- P1: Rotação entre k1 e k2



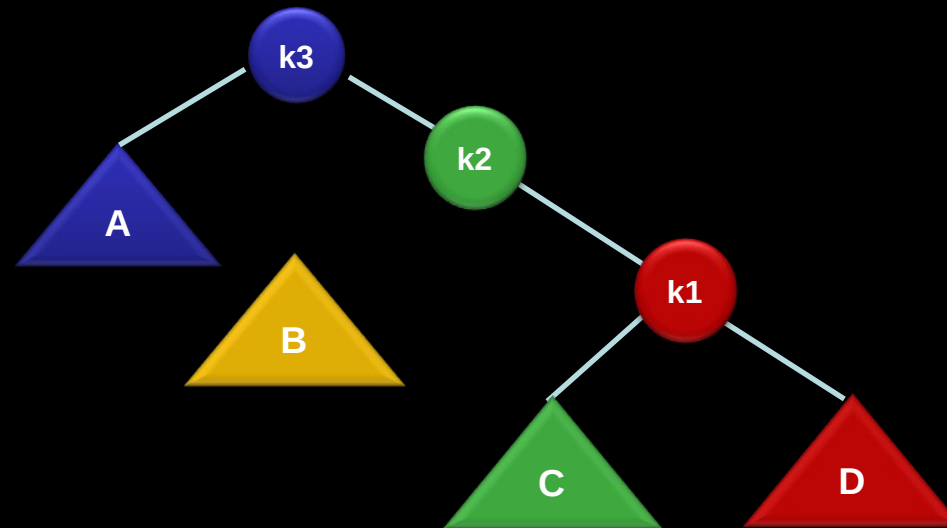
Rotação dupla à esquerda

- P2: Rotação entre k2 e k3



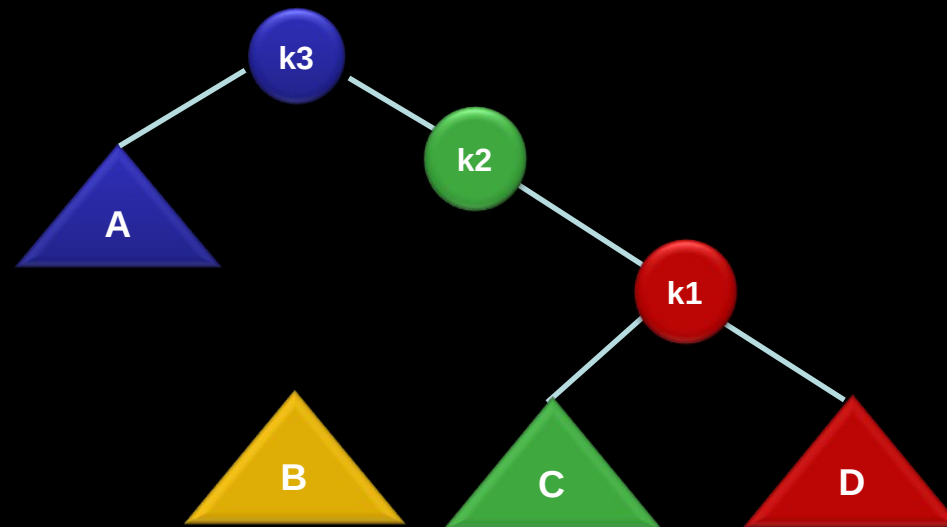
Rotação dupla à esquerda

- P2: Rotação entre k2 e k3



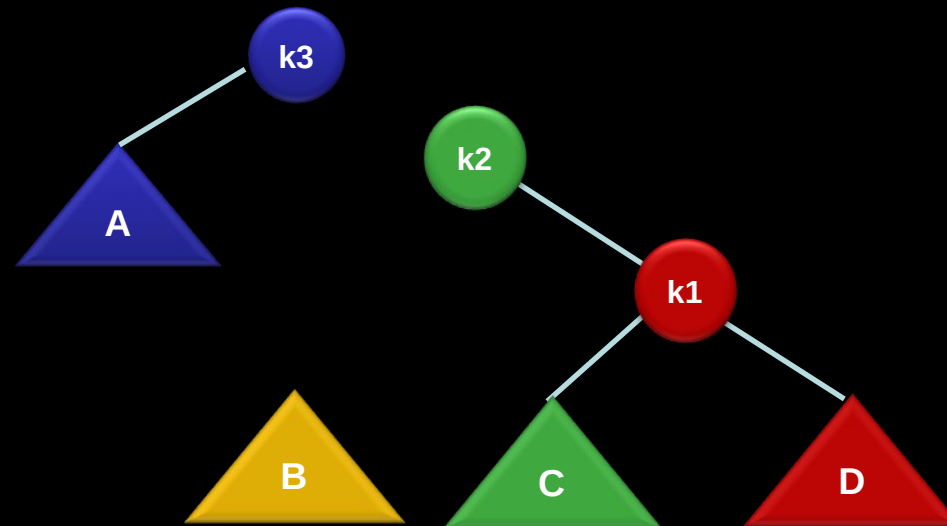
Rotação dupla à esquerda

- P2: Rotação entre k2 e k3



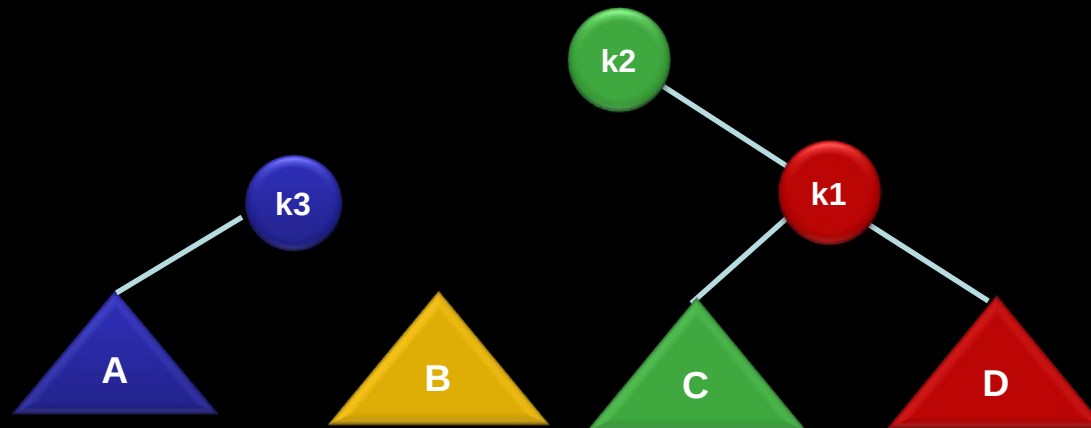
Rotação dupla à esquerda

- P2: Rotação entre k2 e k3



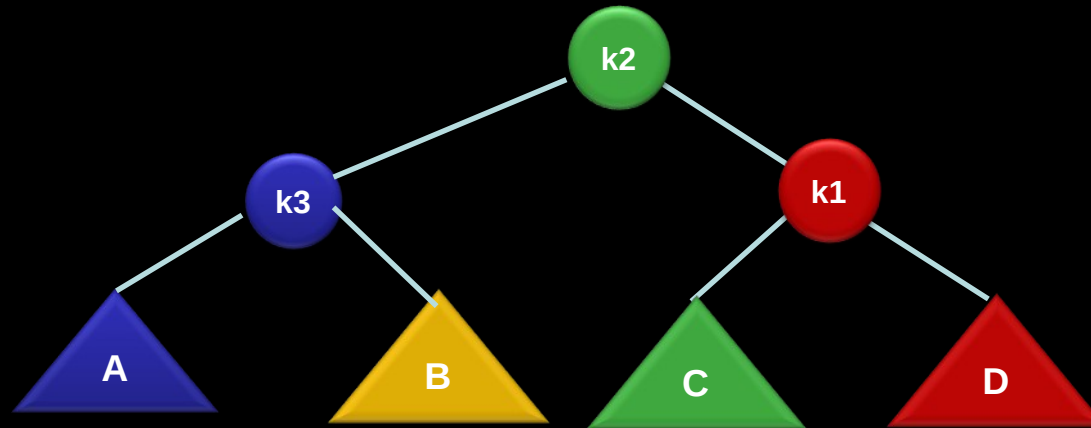
Rotação dupla à esquerda

- P2: Rotação entre k2 e k3



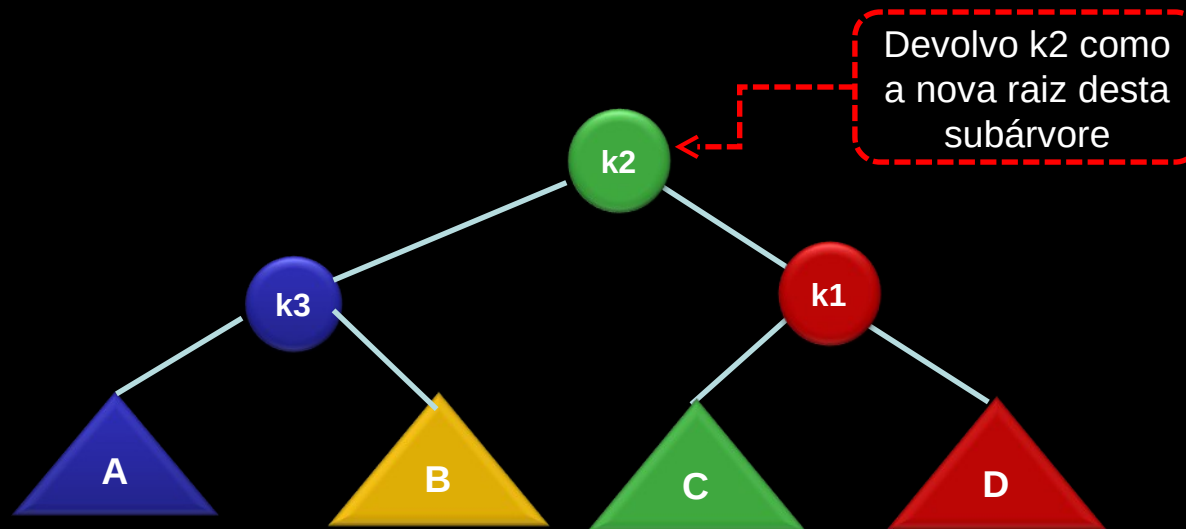
Rotação dupla à esquerda

- P2: Rotação entre k2 e k3



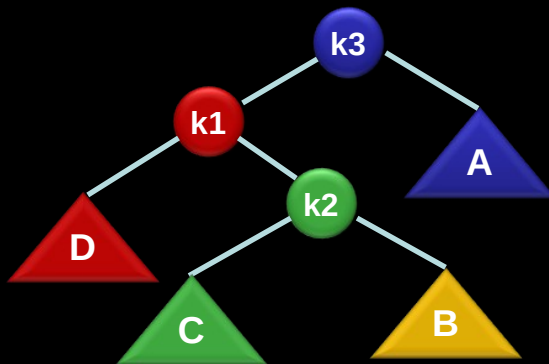
Rotação dupla à esquerda

- P2: Rotação entre k2 e k3



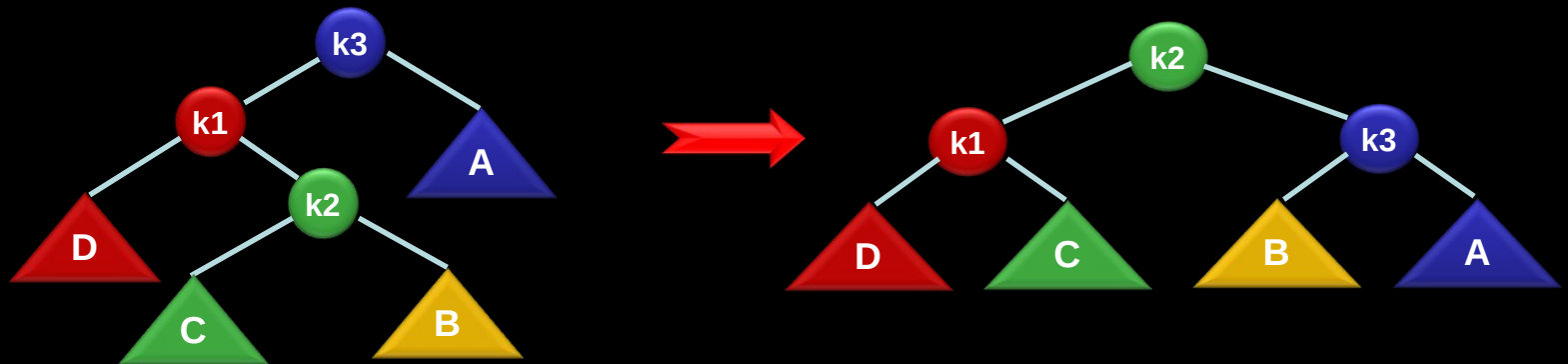
Rotação dupla à direita

- Exemplo simétrico ao anterior.



Rotação dupla à direita

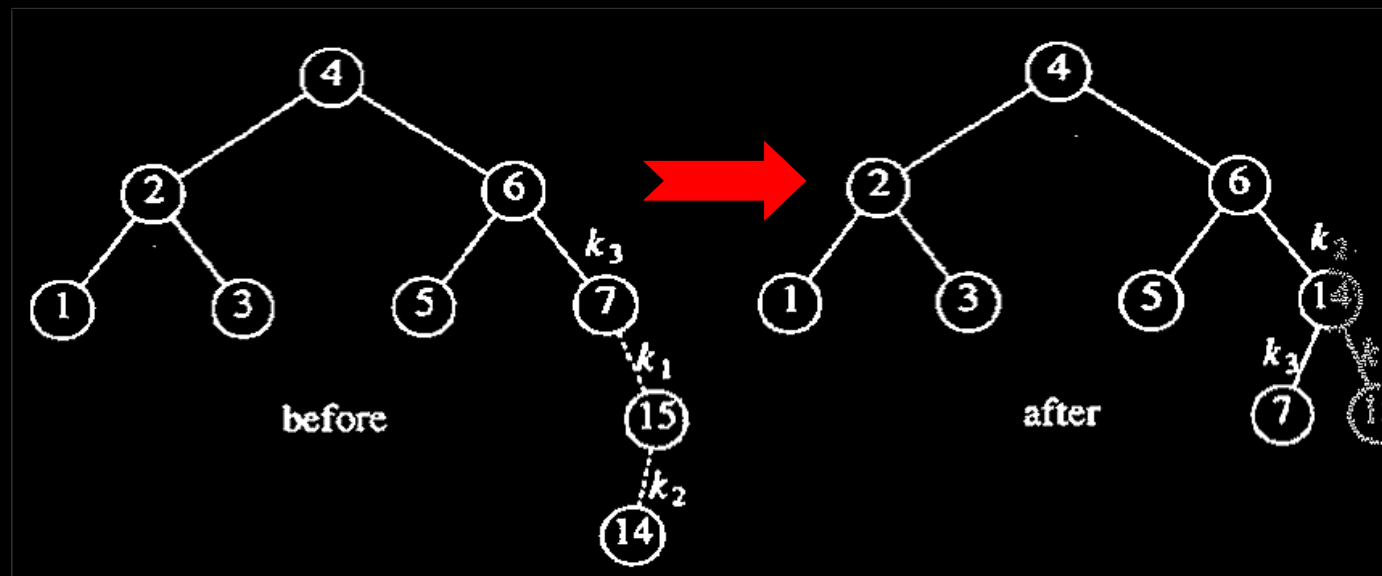
- Exemplo simétrico ao anterior.



Inclusão do elemento 14 com rotação dupla

No exemplo, a rotação **direita-esquerda** envolve o **7**, o **15** e o **14**:

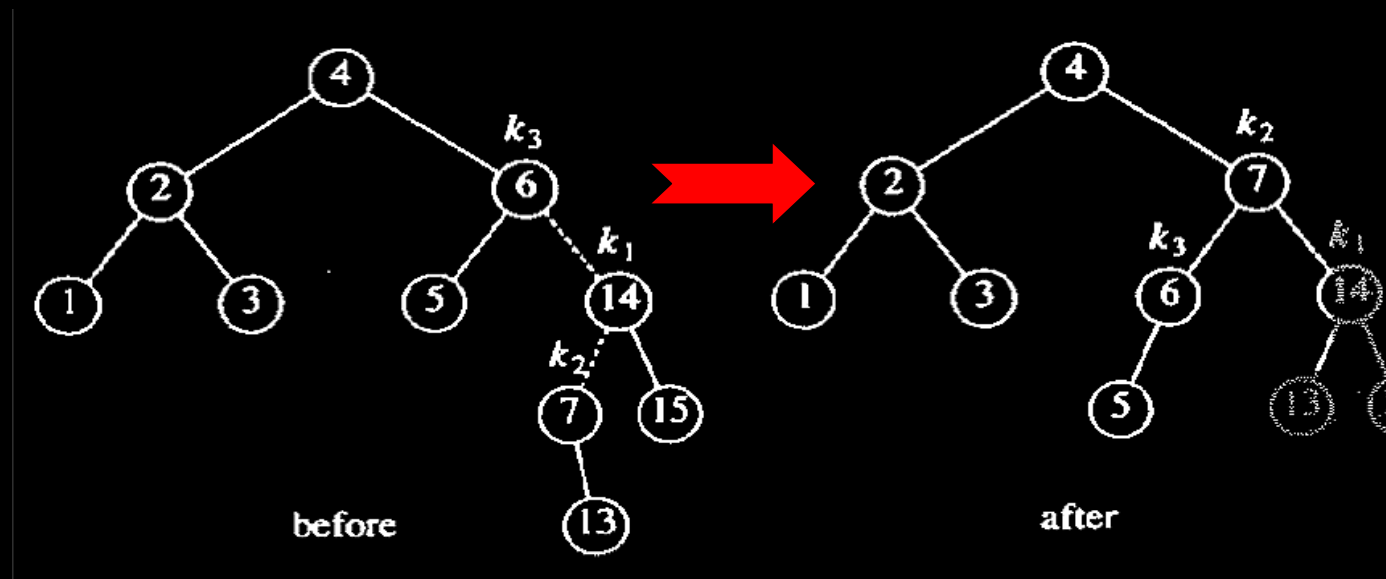
- k_3 é o nodo de chave **7**, k_1 o de chave **15** e k_2 o de chave **14**;
- primeiro rotacionamos **14-15** à direita, depois **7-14** à esquerda.



Inclusão do elemento 13 com rotação dupla

Novamente uma rotação direita-esquerda:

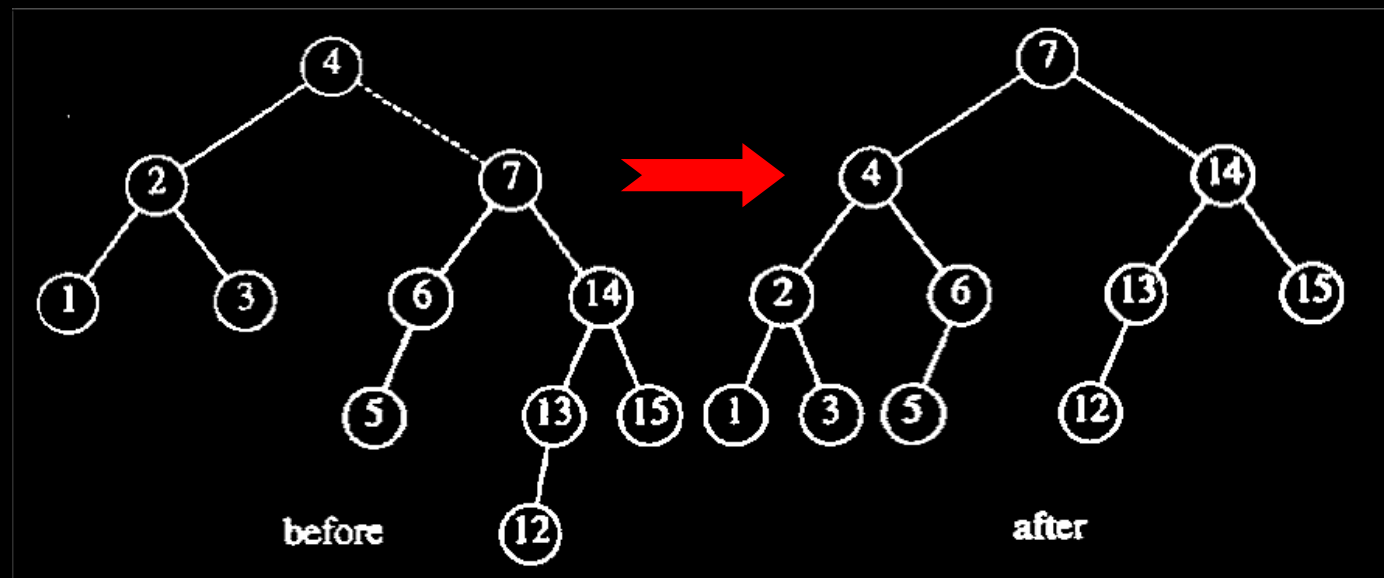
- desta vez a rotação envolverá 6 (k_3), 14 (k_1) e 7 (k_2).



Inclusão do elemento 12

A inclusão do elemento **12** provoca um desequilíbrio na raiz:

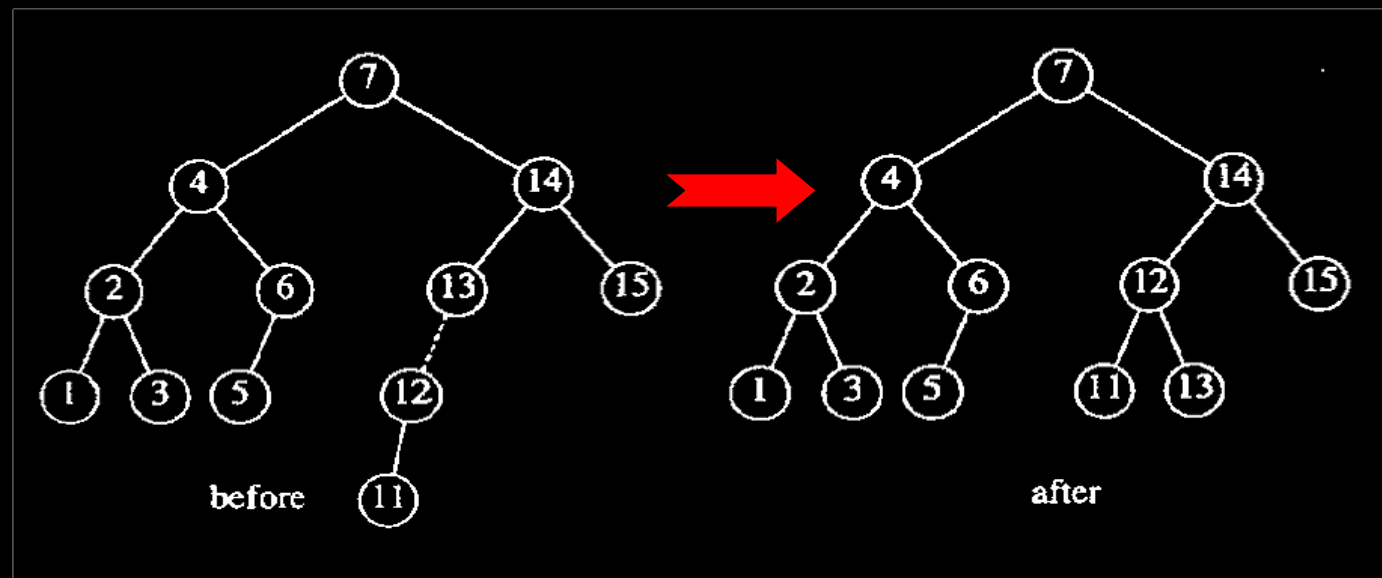
- como **12** não está entre **4** e **7**, sabemos que uma rotação simples vai funcionar.



Inclusão do elemento 11

A inclusão do elemento **11** provoca um desequilíbrio logo embaixo:

- como **11** não está entre **12** e **13**, sabemos que uma rotação simples vai funcionar.



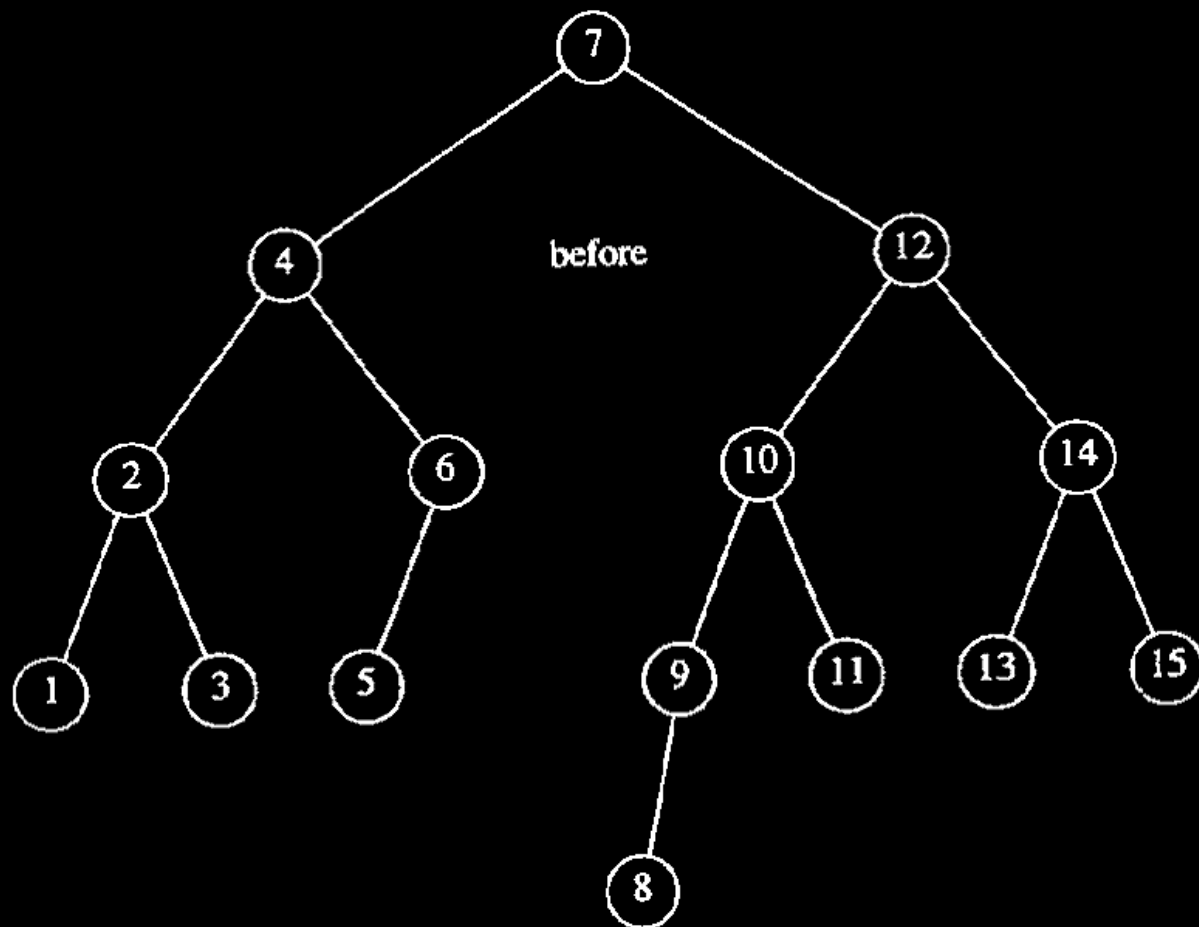
Inclusão dos elementos 10, 9 e 8

Os elementos 10 e 9 serão inseridos com rotação simples, e o elemento 8 sem rotação alguma.

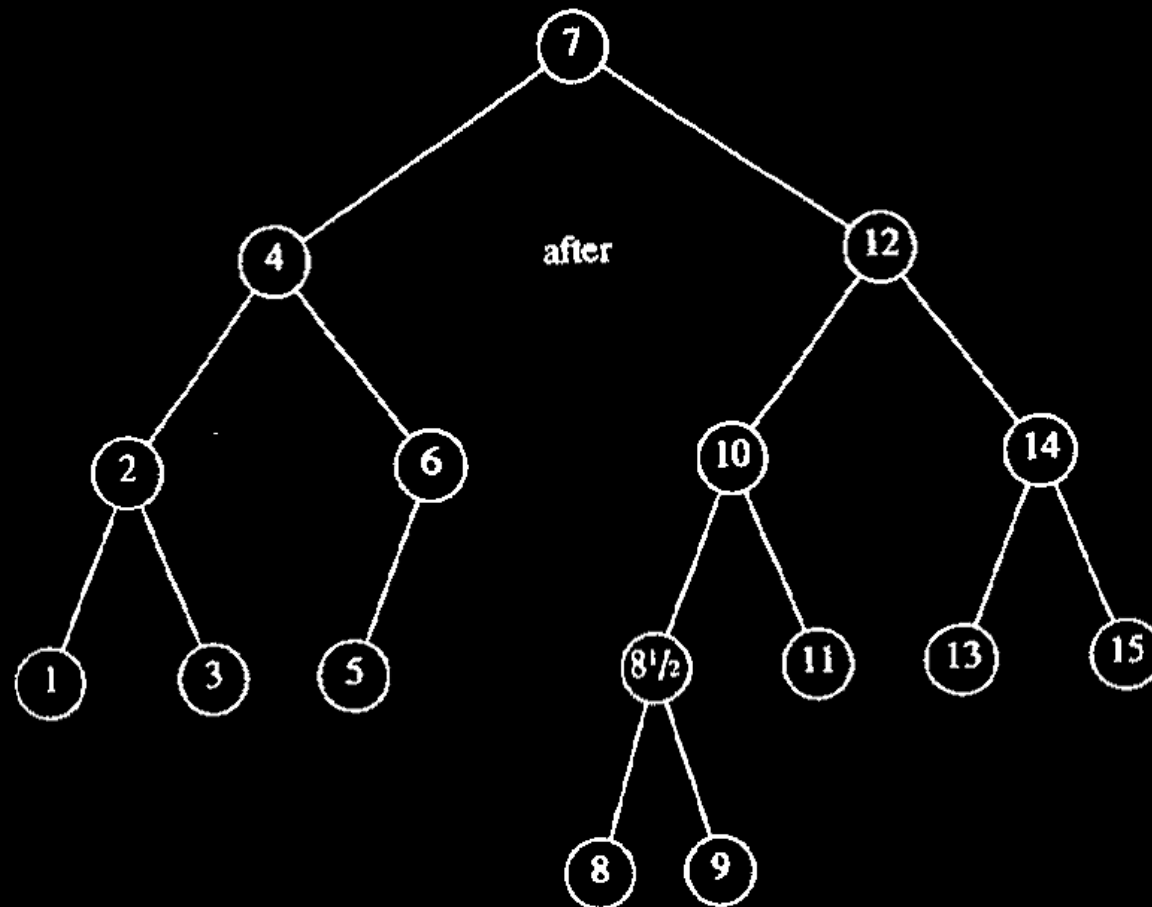
- Tente em casa

A árvore resultante será vista a seguir.

Caso interessante: inclusão do elemento 8.5



Temos uma rotação dupla que parece ser uma simples



Rotação Dupla à Esquerda

- Função chamada somente se k3 possuir um filho à direita e se o filho à direita de k3 possuir um filho à esquerda. Ela realiza a rotação e atualiza as alturas das subárvores rotacionadas.

```
nodoAVL *dup_roda_esq(k3 : *nodoAVL)
início
    /* Rotacione entre k1 e k2 */
    k3->dir <- simp_roda_dir( k3-> dir );
    /* Rotacione entre k3 e k2 */
    retorne (simp_roda_esq( k3 ));
fim
```

- Da mesma forma que com a rotação simples, a *rotação dupla à esquerda* é a operação simétrica da rotação à esquerda e é facilmente implementada.

Inserção em Árvore AVL

- A inserção em uma árvore AVL é necessariamente efetuada recursivamente,
 - ao se retornar de cada nível (chamada recursiva), a subárvore onde ocorreu a inserção deve ser rebalanceada se necessário.

Inserção em Árvore AVL

```
nodoAVL *inserçãoAVL(T* info , *nodoAVL arv , *nodoAVL pai)
variáveis
    arv_rodada : *nodoAVL;
início
    se (arv é NULO) então /* Folha: aloca novo nodo */
        arv <- aloca novo nodo;
        se (arv é NULO) então retorne ERRO;
        arv ->info <- info; arv ->alt <- 0;
        arv ->esq <- NULO; arv ->dir <- NULO;
    senão
        se (info < arv ->info) então
            arv ->esq <- inserçãoAVL(info , arv ->esq , arv);
            se (( altura(arv ->esq) - altura(arv ->dir) > 1)
                se (info < arv ->esq ->info) então
                    arv_rodada <- simp_roda_esq(arv);
                senão arv_rodada <- dup_roda_esq(arv);
            fim se
            se (pai ->esq = arv) então pai ->esq <- arv_rodada;
            senão pai ->dir <- arv_rodada;
            fim se
        senão
            arv ->alt <- max( altura(arv ->esq), altura(arv ->dir)) + 1;
        fim se
    senão // (continua)
```

Inserção em Árvore AVL

```
nodoAVL *inserçãoAVL(T* info , *nodoAVL arv , *nodoAVL pai)
variáveis
    arv_rodada : *nodoAVL;
início
    se (arv é NULO) então /* Folha: aloca novo nodo */
        arv <- aloca novo nodo;
        se (arv é NULO) então retorne ERRO;
        arv ->info <- info; arv ->alt <- 0;
        arv ->esq <- NULO; arv ->dir <- NULO;
    senão
        se (info < arv ->info) então
            arv ->esq <- inserçãoAVL(info , arv ->esq , arv);
            se (( altura(arv ->esq) - altura(arv ->dir) > 1)
                se (info < arv ->esq ->info) então
                    arv_rodada <- simp_roda_esq(arv);
                senão arv_rodada <- dup_roda_esq(arv);
            fim se
            se (pai ->esq = arv) então pai ->esq <- arv_rodada;
            senão pai ->dir <- arv_rodada;
            fim se
        senão
            arv ->alt <- max( altura(arv ->esq), altura(arv ->dir)) + 1;
        fim se
    senão // (continua)
```

Inserção em Árvore AVL

```
nodoAVL *inserçãoAVL(T* info , *nodoAVL arv , *nodoAVL pai)
variáveis
    arv_rodada : *nodoAVL;
início
    se (arv é NULO) então /* Folha: aloca novo nodo */
        arv <- aloca novo nodo;
        se (arv é NULO) então retorne ERRO;
        arv ->info <- info; arv ->alt <- 0;
        arv ->esq <- NULO; arv ->dir <- NULO;
    senão
        se (info < arv ->info) então
            arv ->esq <- inserçãoAVL(info , arv ->esq , arv);
            se (( altura(arv ->esq) - altura(arv ->dir) > 1) // Rotaciona?
                se (info < arv ->esq ->info) então
                    arv_rodada <- simp_roda_esq(arv);
                senão arv_rodada <- dup_roda_esq(arv);
            fim se
            se (pai ->esq = arv) então pai ->esq <- arv_rodada;
            senão pai ->dir <- arv_rodada;
            fim se
        senão
            arv ->alt <- max( altura(arv ->esq), altura(arv ->dir)) + 1;
        fim se
    senão // (continua)
```

Inserção em Árvore AVL

```
nodoAVL *inserçãoAVL(T* info , *nodoAVL arv , *nodoAVL pai)
variáveis
    arv_rodada : *nodoAVL;
início
    se (arv é NULO) então /* Folha: aloca novo nodo */
        arv <- aloca novo nodo;
        se (arv é NULO) então retorne ERRO;
        arv ->info <- info; arv ->alt <- 0;
        arv ->esq <- NULO; arv ->dir <- NULO;
    senão
        se (info < arv ->info) então
            arv ->esq <- inserçãoAVL(info , arv ->esq , arv);
            se (( altura(arv ->esq) - altura(arv ->dir) > 1)
                se (info < arv ->esq ->info) então // Qual rotação?
                    arv_rodada <- simp_roda_esq(arv);
                senão arv_rodada <- dup_roda_esq(arv);
                fim se
            se (pai ->esq = arv) então pai ->esq <- arv_rodada;
            senão pai ->dir <- arv_rodada;
            fim se
        senão
            arv ->alt <- max( altura(arv ->esq), altura(arv ->dir)) + 1;
        fim se
    senão // (continua)
```

Inserção em Árvore AVL

```
nodoAVL *inserçãoAVL(T* info , *nodoAVL arv , *nodoAVL pai)
variáveis
    arv_rodada : *nodoAVL;
início
    se (arv é NULO) então /* Folha: aloca novo nodo */
        arv <- aloca novo nodo;
        se (arv é NULO) então retorne ERRO;
        arv ->info <- info; arv ->alt <- 0;
        arv ->esq <- NULO; arv ->dir <- NULO;
    senão
        se (info < arv ->info) então
            arv ->esq <- inserçãoAVL(info , arv ->esq , arv);
            se (( altura(arv ->esq) - altura(arv ->dir) > 1)
                se (info < arv ->esq ->info) então
                    arv_rodada <- simp_roda_esq(arv);
                senão arv_rodada <- dup_roda_esq(arv);
            fim se // Acerta o pai
            se (pai ->esq = arv) então pai ->esq <- arv_rodada;
            senão pai ->dir <- arv_rodada;
            fim se
        senão
            arv ->alt <- max( altura(arv ->esq), altura(arv ->dir)) + 1;
        fim se
    senão // (continua)
```

Inserção em Árvore AVL

```
nodoAVL *inserçãoAVL(T* info , *nodoAVL arv , *nodoAVL pai)
variáveis
    arv_rodada : *nodoAVL;
início
    se (arv é NULO) então /* Folha: aloca novo nodo */
        arv <- aloca novo nodo;
        se (arv é NULO) então retorne ERRO;
        arv ->info <- info; arv ->alt <- 0;
        arv ->esq <- NULO; arv ->dir <- NULO;
    senão
        se (info < arv ->info) então
            arv ->esq <- inserçãoAVL(info , arv ->esq , arv);
            se (( altura(arv ->esq) - altura(arv ->dir) > 1)
                se (info < arv ->esq ->info) então
                    arv_rodada <- simp_roda_esq(arv);
                senão arv_rodada <- dup_roda_esq(arv);
            fim se
            se (pai ->esq = arv) então pai ->esq <- arv_rodada;
            senão pai ->dir <- arv_rodada;
            fim se
        senão // Ajusta a altura
            arv ->alt <- max( altura(arv ->esq), altura(arv ->dir)) + 1;
        fim se
    senão // (continua)
```

Inserção em Árvore AVL

```
senão //(continuado de cima)
  se (info > arv ->info) então
    /* caso simétrico para árvore direita */
    arv ->dir <- inserçãoAVL(info , arv ->dir , arv);
    se (( altura(arv ->dir) - altura(arv ->esq) > 1)
      se (info < arv ->dir ->info) então
        arv_rodada <- simp_roda_dir(arv);
      senão
        arv_rodada <- dup_roda_dir(arv);
      fim se
      se (pai ->dir = arv) então pai ->dir <- arv_rodada;
      senão pai ->esq <- arv_rodada;
      fim se
    senão
      arv ->alt <- max( altura(arv ->esq), altura(arv ->dir)) + 1;
    fim se
  senão
    retorne ERRO: "chave já está na árvore"
  fim se
fim se
retorne arv;
fim
```

Deleção em Árvore AVL

- Busque a chave a deletar na árvore;
- Delete utilizando o algoritmo **recursivo** de Deleção da Árvore Binária de Busca;
- Modifique para:
 - Logo após retorno da chamada recursiva atualizar & verificar alturas das subárvores filhas;
 - Aplicar "regra do zigue-zague" se necessário.

Deleção em Árvore AVL

- "regra do zigue-zague":
 - Olhar subárvore que desequilibrou e seus filhos;
- Desequilíbrio é à esquerda:
 - Sub-subárvore esquerda é a mais funda?
 - Esquerda-esquerda: rotação simples à esquerda;
 - Sub-subárvore direita é a mais funda?
 - Esquerda-direita: rotação dupla à esquerda;
- Desequilíbrio é à direita:
 - Sub-subárvore direita é a mais funda?
 - Direita-direita: rotação simples à direita
 - Sub-subárvore esquerda é a mais funda?
 - Direita-esquerda: rotação dupla à direita